Governance Without TERM Token Veto Configuration Proof

1. High Level Conclusion

```
Branch 1 (the slow path, for real proposals)
  ProposerSafe (5/11) ------module------> Delay Modifier ------module------> OwnerlessSafe (9/9) ------DEVOPS_ROLE------> Term Protocol
  (+ ConfigLockGuard)                     (+ PauseGuard)                  (+ ConfigLockGuard)

Direct path (all nine OwnerlessSafe signers together: no delay, veto or pause)
  OwnerlessSafe (9/9) ------DEVOPS_ROLE------> Term Protocol

Branch 2 (the DelayOwnerSafe's veto path, whose only power is to veto)
  DelayOwnerSafe (5/11) ------owner------> Delay Modifier
  (+ ConfigLockGuard)

Pause controls
  PauseSafe (2/9) --pause-->               PauseGuard
  ProposerSafe   --unpause / setPauser-->  PauseGuard
```

Trust assumption: Despite its name, the OwnerlessSafe has nine Term multisig holders as owners and a threshold of 9, high enough that its direct path has more friction than going through the Delay Modifier.

The conclusions drawn from these proofs establish the following propositions:

1. The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe, and only they, may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard, provided DEVOPS_ROLE method calls from the ProposerSafe are not vetoed, and are executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.
2. Only the Term multisig holders on the DelayOwnerSafe may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier, and 3. the txCooldown constant for the Delay Modifier.
3. Only the Term multisig holders on the PauseSafe may pause, and only the Term multisig holders on the ProposerSafe may unpause, the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.

4. The owners for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe. The fallbackHandler, modules, guard, module guard for the OwnerlessSafe can be changed only by the Term multisig holders on the ProposerSafe. The following: 1. the fallbackHandler, modules, guard for the ProposerSafe and DelayOwnerSafe, and 2. the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by anyone.
5. The owners, fallbackHandler, modules, guard for the PauseSafe can be changed only by the Term multisig holders on the PauseSafe. The pauser for the PauseGuard can be changed only by the Term multisig holders on the ProposerSafe. The admin for the PauseGuard cannot be changed by anyone.

Only Term multisig holders can execute DEVOPS_ROLE methods, and this cannot be changed other than through Term multisig approval.

2. Evidence Classification

This document uses two classes of evidence to establish the security and functional properties of the governance configuration:

- **Formal Verification (FV) evidence** establishes that, assuming the stated configuration, the relevant contracts enforce the claimed security and functional properties. These properties are proved with the Certora Prover, a formal verification tool. For the guards (PauseGuard and ConfigLockGuard), the proofs run on the contract code that was deployed, the code at commit `e9f3d20f4d16af7ab7a9fdf773e4fabd4cde3353` on `main`. The proofs cover privileged entry points, authorized callers, configuration immutability, the absence of alternative execution paths, and the intended functionality. In the tables below, FV evidence is labeled **FV —** and names each Certora rule; ✅ marks a rule that passes.
- **Deployment evidence** states the configuration the 2.1.0 deployment must have for the formal verification evidence to apply: owners, modules, guards, thresholds, roles, and other relevant state. 2.1.0 is a proposal, not a live deployment, so none of it has been read on chain yet. In the tables below, deployment evidence is labeled **On-chain —**; ⏳ marks a requirement to verify against on-chain state, at a stated block, once the 2.1.0 contracts are deployed.

These two classes of evidence are complementary. Formal verification evidence does not by itself establish that the deployed contracts have the configuration being proved, and deployment evidence does not by itself establish that the deployed configuration enforces the claimed security and functional properties.

Safe versions. The ProposerSafes and OwnerlessSafes already deployed run GnosisSafeL2 v1.3.0 or SafeL2 v1.4.1, and any deployed from now on will run SafeL2 v1.5.0. No DelayOwnerSafe or PauseSafe exists yet, so each will be deployed on SafeL2 v1.5.0. The formal verification evidence therefore covers the ProposerSafe and OwnerlessSafe on GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 and SafeL2 v1.5.0, and the DelayOwnerSafe and PauseSafe on SafeL2 v1.5.0 only. Each version is the L2 build of the Safe, so the rules run against that build.

Configuration items marked ⏳ (pending deployment) describe the configuration of the proposed 2.1.0 deployment and are not established yet. Each must be verified against on-chain state once the 2.1.0 contracts are deployed.

3. Evidence

Proposition 1 — The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe, and only they, may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and

txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard, provided DEVOPS_ROLE method calls from the ProposerSafe are not vetoed, and are executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.

#	Statement	Evidence
P1.1	Only the OwnerlessSafe may execute DEVOPS_ROLE methods.	**On-chain — The OwnerlessSafe is the only holder of the DEVOPS_ROLE AccessControl role required for DEVOPS_ROLE methods:** • The DEVOPS_ROLE methods are those listed in the DEVOPS_ROLE row of Term Finance's Access Controls page: `_authorizeUpgrade` on all contracts, `pauseDeploying` and `unpauseDeploying` on TermInitializer, `updateTreasuryAddress` and `updateProtocolReserveAddress` on TermController, and `addNewTokenPriceFeed` and `removeTokenPriceFeed` on TermPriceConsumerV3. • ⏳ pending deployment. Each of the six Term Protocol contracts (TermController, TermEventEmitter, TermPriceConsumerV3, TermRepoDeployerFactory, TermInitializer and TermDiamond) must grant `DEVOPS_ROLE` to the OwnerlessSafe and to no other address: in each contract's log history, the only `RoleGranted` naming `DEVOPS_ROLE` must be for the OwnerlessSafe, with no `RoleRevoked` and no `RoleAdminChanged`. • Today the six contracts grant `DEVOPS_ROLE` only to the 2.0.0 OwnerlessSafe, and nobody holds `DEFAULT_ADMIN_ROLE` on them (below), so `DEVOPS_ROLE` cannot be granted to a newly deployed Safe. The 2.1.0 OwnerlessSafe must either be that Safe or come with Term Protocol contracts that grant it `DEVOPS_ROLE`. • The other contracts with `_authorizeUpgrade` are proxies deployed against implementations, such as each term's TermAuction. Term's deploy system deploys every new proxy with `DEVOPS_ROLE` assigned only to the OwnerlessSafe. The deploy system runs off chain, so this is a deployment fact, not an on-chain read. **On-chain — Nobody can grant DEVOPS_ROLE to another address that is not the OwnerlessSafe:** • ⏳ pending deployment. On TermController, TermEventEmitter, TermPriceConsumerV3, TermInitializer and TermRepoDeployerFactory, `getRoleAdmin(DEVOPS_ROLE)` must be `DEFAULT_ADMIN_ROLE` (`0x00`), so only a holder of `DEFAULT_ADMIN_ROLE` can call `grantRole(DEVOPS_ROLE, …)`. • ⏳ pending deployment. No address may hold `DEFAULT_ADMIN_ROLE` on any of the six contracts: their log histories must have no `RoleGranted` for it. So nobody can grant `DEVOPS_ROLE` on those five contracts.
P1.2	Only Modules of OwnerlessSafe and Owners of OwnerlessSafe can interact with the OwnerlessSafe, except `execTransaction`, which anyone can call only if Owners of OwnerlessSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	**FV — Callers can only interact with the Safe through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setup`, `simulateAndRevert`, `fallback` and `receive`, plus `requiredTxGas` on GnosisSafeL2 v1.3.0 and `setModuleGuard` on SafeL2 v1.5.0:** • SE130L2-1 / SE141L2-1 / SE150L2-1 `safeEntryPointsAreAllAccountedFor` ✅ passes: SafeL2 v1.4.1 has no write functions besides `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setup`, `simulateAndRevert`, `fallback` and `receive` (the Prover checks `fallback` and `receive` as one entry), GnosisSafeL2 v1.3.0 has these and `requiredTxGas`, and SafeL2 v1.5.0 has these and `setModuleGuard`. Any unsupported call lands in `fallback`. Each blob below covers some of these entry points, and together they cover all of them: sixteen on SafeL2 v1.4.1 and seventeen on the other two. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Safe:** • SE130L2-3 / SE141L2-3 / SE150L2-3 `onlyEnabledModulesCanCallModuleExec` ✅ passes: only an enabled module (Modules of Safe) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • SE130L2-4 / SE141L2-4 / SE150L2-4 `enabledModuleCanMakeTheSafeAct` ✅ passes: an enabled module can call them

#	Statement	Evidence
		directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. So SE130L2-3 / SE141L2-3 / SE150L2-3 is not vacuous. **FV — `approveHash` is only callable by Owners of Safe:** • SE130L2-8 / SE141L2-8 / SE150L2-8 `onlyOwnersCanApproveHashes` ✅ passes: in any call, an approval is only recorded when the caller is an owner (Owners of Safe) recording its own approval. So a call to `approveHash` from anyone else records nothing. An owner's recorded approval then counts as that owner's signature in `execTransaction`. **FV — `execTransaction` is callable by anyone, but only executes once a threshold of Owners of Safe have approved:** • SE130L2-5 / SE141L2-5 / SE150L2-5 `execTransactionRunsOnlyAfterTheSignatureCheck` ✅ passes: whoever the caller is, `execTransaction` only runs once the signature check has passed for that exact transaction, asking for as many signatures as the threshold. The rule places no restriction on the caller. • SE130L2-12 / SE141L2-12 / SE150L2-12 `eachSignatureAcceptsANewApprovingOwner` ✅ passes: each signature the check accepts comes from a different owner (Owners of Safe) who approved the transaction. For an approval made through `approveHash`, SE130L2-8 / SE141L2-8 / SE150L2-8 ensures the owner recorded it itself. So with SE130L2-5 / SE141L2-5 / SE150L2-5, the Safe acts through `execTransaction` only once as many different owners as the threshold have approved. • SE130L2-6 / SE141L2-6 / SE150L2-6 `ownersCanMakeTheSafeAct` ✅ passes: once the signature threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners (Owners of Safe). So SE130L2-5 / SE141L2-5 / SE150L2-5 is not vacuous. **FV — `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler` and, on SafeL2 v1.5.0, `setModuleGuard` are only callable from Modules of Safe or Owners of Safe:** • SE130L2-7 / SE141L2-7 / SE150L2-7 `settingsOnlyChangeWhenTheSafeCallsItself` ✅ passes: in any call, the owners, threshold, modules, guard and fallbackHandler, and on SafeL2 v1.5.0 the module guard, only change when the caller is the Safe itself. So a call to any of these functions (eight, or nine on SafeL2 v1.5.0) from anyone else changes nothing. • SC130L2-4 / SC141L2-4 / SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included (on SafeL2 v1.5.0, while no fallbackHandler is set, P1.3), a setting only changes through an enabled module's (Modules of Safe) `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. On GnosisSafeL2 v1.3.0 and SafeL2 v1.4.1, a fallbackHandler is treated as the Safe itself, so it cannot reach these functions even then. By the `execTransactionFromModule` and `execTransaction` blobs above, those paths are open only to Modules of Safe and to a threshold of Owners of Safe. • SC130L2-1 / SC141L2-1 / SC150L2-1 `moduleCanChangeSettings`, SC130L2-2 / SC141L2-2 / SC150L2-2 `moduleCanChangeSettingsWithReturnData` ✅ pass: for each function above, a module's (Modules of Safe) `execTransactionFromModule`, and likewise its `execTransactionFromModuleReturnData`, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC130L2-3 / SC141L2-3 / SC150L2-3 `ownersCanChangeSettings` ✅ passes: for each function above, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. Together with SC130L2-1 / SC141L2-1 / SC150L2-1 and SC130L2-2 / SC141L2-2 / SC150L2-2, this shows the restriction is not achieved by nothing working. **FV — `setup` always reverts after deploy:** • SE130L2-9 / SE141L2-9 / SE150L2-9 `setupAlwaysRevertsAfterSetup` ✅ passes: once the Safe is set up, `setup` always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. • The OwnerlessSafe is set up: its threshold is 9 and it has nine owners (P1.4).

#	Statement	Evidence
		FV — `simulateAndRevert` always reverts: • SE130L2-11 / SE141L2-11 / SE150L2-11 `simulateAndRevertAlwaysReverts` ✅ passes: anyone can call `simulateAndRevert`, and it does make a delegatecall, but it always reverts afterwards, so none of its effects persist. **FV — `requiredTxGas` always reverts (GnosisSafeL2 v1.3.0 only):** • SE130L2-10 `requiredTxGasAlwaysReverts` ✅ passes (GnosisSafeL2 v1.3.0 only): anyone can call `requiredTxGas`, and it does run the transaction it is given, but it always reverts afterwards, so none of its effects persist. SafeL2 v1.4.1 and SafeL2 v1.5.0 have no `requiredTxGas`. **FV — `fallback` is callable by anyone but does not change state, assuming the fallbackHandler is `address(0)`:** • SE130L2-21 / SE141L2-21 / SE150L2-21 `noHandlerFallbackChangesNoState` ✅ passes: with no handler set, `fallback` changes no state at all. It writes no storage, makes no call of any kind (call, delegatecall, staticcall or callcode) and creates or destroys no contract, and a `fallback` call that succeeds carries no ETH, so not even the Safe's balance changes. • SE130L2-23 / SE141L2-23 / SE150L2-23 `storageHookFires`, SE130L2-24 / SE141L2-24 / SE150L2-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE130L2-21 / SE141L2-21 / SE150L2-21's checks do detect storage writes and calls. • SE130L2-22 / SE141L2-22 / SE150L2-22 `noHandlerFallbackSucceeds` ✅ passes: with no handler set, some call to `fallback` succeeds, so SE130L2-21 / SE141L2-21 / SE150L2-21 is not vacuous. **FV — `receive` is callable by anyone but does not change state:** • SE130L2-25 / SE141L2-25 / SE150L2-25 `receiveChangesNoState` ✅ passes: whatever the handler, a call that runs `receive` changes no state beyond the ETH it is sent. It writes no storage, makes no call of any kind and creates or destroys no contract. • Here, state means the Safe's storage, which holds its owners, threshold, modules, guard, fallbackHandler, nonce and approvals. The ETH sent is added to the Safe's balance, which is not part of that storage. • SE130L2-23 / SE141L2-23 / SE150L2-23 `storageHookFires`, SE130L2-24 / SE141L2-24 / SE150L2-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE130L2-25 / SE141L2-25 / SE150L2-25's checks do detect storage writes and calls. • SE130L2-26 / SE141L2-26 / SE150L2-26 `receiveRuns` ✅ passes: some call to the Safe runs `receive` and succeeds, so SE130L2-25 / SE141L2-25 / SE150L2-25 is not vacuous. **On-chain — The OwnerlessSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0, the code the SE and SC rules above check:** • ⏳ pending deployment. The OwnerlessSafe must run GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0: storage slot 0 (the proxy's `singleton`) must hold the canonical singleton of one of them, and `VERSION()` must return that version (`1.3.0`, `1.4.1` or `1.5.0`). `VERSION()` alone does not tell an L2 build from the non-L2 build of the same version; the singleton does. • So the rules above, which pass on all three versions, hold for the OwnerlessSafe. • Assumes the OwnerlessSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0, the versions these rules check.
P1.3	OwnerlessSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard	**On-chain — OwnerlessSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:12` / `:14` / `:13` in GnosisSafeL2 v1.3.0 / SafeL2 v1.4.1 / SafeL2 v1.5.0) on the OwnerlessSafe must hold `address(0)`.

#	Statement	Evidence
	constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	• Assumes the OwnerlessSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.2), each of which keeps the fallbackHandler in this slot.
P1.4	`Owners of OwnerlessSafe = Term multisig holders on OwnerlessSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — Owners of OwnerlessSafe are nine Term multisig holders, and all nine must approve each transaction:** • ⏳ pending deployment. The OwnerlessSafe must run GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (`VERSION()` and storage slot 0, as in P1.2), with threshold 9 (`getThreshold`) and nine owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the OwnerlessSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.2). • The identities of the nine key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P1.5	`Modules of OwnerlessSafe = Delay Modifier`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — The Modules of OwnerlessSafe list contains only the Delay Modifier:** • ⏳ pending deployment. `getModulesPaginated` on the OwnerlessSafe must return just the Delay Modifier, and report that the list ends there. • Assumes the OwnerlessSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.2).
P1.6	Only the Modules of the Delay Modifier and the Owner of the Delay Modifier can interact with the Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by the Modules of the Delay Modifier or the Owner of the Delay Modifier.	**FV — Callers can only interact with the Delay Modifier through `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership`, `renounceOwnership`, `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `executeNextTx`, `skipExpired` and `setUp`:** • DM-13 `delayWriteFunctionsAreTheKnownFifteen` ✅ passes: the Delay Modifier has no fallback and no write functions besides `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership`, `renounceOwnership`, `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `executeNextTx`, `skipExpired` and `setUp`. Each blob below covers some of these, and together they cover all fifteen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Delay Modifier:** • DM-20 `onlyEnabledModulesCanCallExecFromModule` ✅ passes: only an enabled module (Modules of Delay Modifier) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • DM-5 `enabledModuleCanQueue` ✅ passes: an enabled module's call to `execTransactionFromModule` succeeds. So DM-20 is not vacuous. **FV — `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership` are only callable by Owner of Delay Modifier:**

#	Statement	Evidence
		• DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of these ten functions succeeds only when the caller is the owner (Owner of Delay Modifier), and from the same state no two different callers can both succeed. • DM-19 `ownerCanCallEachDelaySetting` ✅ passes: for each of the ten, some call from the owner succeeds. So DM-8 is not vacuous. **FV — `setUp` always reverts after deploy:** • DM-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: nobody can re-run the Delay Modifier's setup after deployment. **FV — `executeNextTx` and `skipExpired` are callable by anyone, but can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier:** • DM-11 `anyoneCanExecuteNextTx`, DM-12 `anyoneCanSkipExpired` ✅ pass: whether a call to `executeNextTx` or `skipExpired` succeeds, and what it changes, does not depend on who the caller is. • DM-17 `outsiderCanExecuteNextTx`, DM-18 `outsiderCanSkipExpired` ✅ pass: a caller that is neither the owner nor a module can execute or skip a queue entry. So DM-11 and DM-12 are not vacuous. • DQ-2 `everyQueuedEntryWasQueuedByModuleOrOwner` ✅ passes: every transaction in the queue was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-3 `executeNextTxRunsOnlyEntriesQueuedByModuleOrOwner` ✅ passes: `executeNextTx` only runs the queue entry at `txNonce`. By DQ-2, that entry was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-4 `skipExpiredSkipsOnlyEntriesQueuedByModuleOrOwner` ✅ passes: every index `skipExpired` steps over is a queue entry, and by DQ-2 it was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-5 `executeNextTxCanRunAQueuedEntry`, DQ-6 `skipExpiredCanSkipAQueuedEntry` ✅ pass: `executeNextTx` can run a queue entry, and `skipExpired` can skip one. So DQ-3 and DQ-4 are not vacuous.
P1.7	Only the Modules of the Delay Modifier can queue transactions to the Delay Modifier. The Owner of the Delay Modifier, the DelayOwnerSafe, cannot queue transactions to the Delay Modifier.	**FV — Only Modules of Delay Modifier can queue transactions to the Delay Modifier:** • DM-4 `queueOnlyGrowsThroughEnabledModules` ✅ passes: over every write function, only an enabled module (Modules of Delay Modifier) can move the queue nonce or write a queue entry. • DM-5 `enabledModuleCanQueue` ✅ passes: an enabled module's call to `execTransactionFromModule` succeeds and moves the queue nonce forward by one. So DM-4 is not vacuous. **On-chain — Owner of Delay Modifier, the DelayOwnerSafe, is not a Module of Delay Modifier:** • ⏳ pending deployment. `owner()` on the Delay Modifier must return the DelayOwnerSafe (P2.3). • ⏳ pending deployment. `isModuleEnabled` on the Delay Modifier must return false for the DelayOwnerSafe. This matches P1.8: the Delay Modifier's only module (Modules of Delay Modifier) is the ProposerSafe. • So, since the DelayOwnerSafe is the owner, by DM-4 the Owner of Delay Modifier cannot queue transactions.
P1.8	`Modules of Delay Modifier = ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — The Modules of Delay Modifier list contains only the ProposerSafe:** • ⏳ pending deployment. `getModulesPaginated` on the Delay Modifier must return just the ProposerSafe, and report that the list ends there.

#	Statement	Evidence
P1.9	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.2. **On-chain — The ProposerSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0, the code P1.2's rules check, so P1.2's evidence applies to the ProposerSafe:** • ⏳ pending deployment. The ProposerSafe must run GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0: storage slot 0 (the proxy's `singleton`) must hold the canonical singleton of one of them, and `VERSION()` must return that version, as in P1.2. • ⏳ pending deployment. The ProposerSafe and the OwnerlessSafe must each have the runtime code of a standard Safe proxy: the proxy that GnosisSafeProxyFactory v1.3.0, SafeProxyFactory v1.4.1 or SafeProxyFactory v1.5.0 deploys. Each of these forwards every call, other than `masterCopy()`, to the singleton in storage slot 0. • So the ProposerSafe, like the OwnerlessSafe, is a standard Safe proxy in front of a mastercopy P1.2's rules check, and those rules, which pass on all three versions, hold for the ProposerSafe too. • Assumes the ProposerSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0, the versions P1.2's rules check.
P1.10	ProposerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — ProposerSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:12` / `:14` / `:13` in GnosisSafeL2 v1.3.0 / SafeL2 v1.4.1 / SafeL2 v1.5.0) on the ProposerSafe must hold `address(0)`. • Assumes the ProposerSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.9), each of which keeps the fallbackHandler in this slot.
P1.11	`Owners of ProposerSafe = Term multisig holders on ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — Owners of ProposerSafe are eleven Term multisig holders, and five of them must approve each transaction:** • ⏳ pending deployment. The ProposerSafe must run GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (`VERSION()` and storage slot 0, as in P1.9), with threshold 5 (`getThreshold`) and eleven owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the ProposerSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.9). • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P1.12	`Modules of ProposerSafe = null`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — The Modules of ProposerSafe list is empty:** • ⏳ pending deployment. `getModulesPaginated` on the ProposerSafe must return an empty list. • Assumes the ProposerSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.9).

#	Statement	Evidence
P1.13	The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe may execute DEVOPS_ROLE methods, provided DEVOPS_ROLE method calls from the ProposerSafe are not vetoed, and are executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.	**FV — The Term multisig holders on the ProposerSafe can execute DEVOPS_ROLE methods by proposing them to the Delay Modifier and executing them after the cooldown, provided they are not vetoed, while the PauseGuard is not paused and before the proposal expires:** • DE130L2-1 / DE141L2-1 / DE150L2-1 `proposerOwnersExecuteDevopsMethodThroughTheDelay` ✅ passes. In one execution: ◦ An owner of the ProposerSafe (a Term multisig holder, P1.11) approves with `approveHash` the ProposerSafe's call to the Delay Modifier's `execTransactionFromModule(<protocol contract>, 0, <DEVOPS_ROLE method call>, Call)`. ◦ Anyone submits it through `execTransaction` with that approval as the signature, so the Delay Modifier queues the proposal. ◦ Nobody vetoes it. Once the cooldown has passed, before the proposal expires, and with the PauseGuard installed and not paused, anyone's `executeNextTx` hands it to the OwnerlessSafe. ◦ The OwnerlessSafe calls the protocol contract. The call passes `onlyRole(DEVOPS_ROLE)` because the OwnerlessSafe holds `DEVOPS_ROLE` (P1.1), and the method runs. • DE130L2-1 / DE141L2-1 / DE150L2-1 sets the ProposerSafe's threshold to 1, so one owner's approval is the whole threshold. SE130L2-6 / SE141L2-6 / SE150L2-6 `ownersCanMakeTheSafeAct` (P1.2) ✅ passes for any threshold, so with the real threshold of 5 (P1.11), five owners' approvals take the same path. • DE130L2-1 / DE141L2-1 / DE150L2-1 assumes the deployed wiring: the ProposerSafe is a module of the Delay Modifier (P1.8), and the Delay Modifier is a module of the OwnerlessSafe (P1.5). The Delay Modifier's target is the OwnerlessSafe and its guard is the PauseGuard. DE130L2-1 / DE141L2-1 / DE150L2-1 also assumes neither Safe has a guard; each has a ConfigLockGuard, which the next two bullets cover. The on-chain blob below covers the parts not shown elsewhere. • The ProposerSafe's ConfigLockGuard lets the proposal through. Its `lockedDelay` is the Delay Modifier (P4.DelayModifier.8), so the proposal, `execTransactionFromModule` on the Delay Modifier, is a plain call to the locked modifier. It is not one of the nine calls the guard locks there (CL130L2-6 to 9 / CL141L2-6 to 9 / CL150L2-6 to 9, CL130L2-24 / CL141L2-24 / CL150L2-24). CL130L2-10 / CL141L2-10 / CL150L2-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the Delay Modifier included, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so DE130L2-1 / DE141L2-1 / DE150L2-1's ProposerSafe step is unchanged. • The OwnerlessSafe's ConfigLockGuard never checks the Delay Modifier's call. Each version consults its guard only in `execTransaction` (GnosisSafeL2 v1.3.0: `GnosisSafe.sol:147–191`; SafeL2 v1.4.1: `Safe.sol:174–218`; SafeL2 v1.5.0: `Safe.sol:149–204`), not in `execTransactionFromModule` (GnosisSafeL2 v1.3.0: `GnosisSafeL2.sol:77–85`, `ModuleManager.sol:61–73`; SafeL2 v1.4.1: `SafeL2.sol:65–73`, `ModuleManager.sol:81–93`; SafeL2 v1.5.0: `ModuleManager.sol:154–163`, which consults only the module guard, `ModuleManager.sol:94–109`). CL130L2-22 / CL141L2-22 / CL150L2-22 `moduleCallBypassesTheGuard` ✅ passes: with ConfigLockGuard installed, an enabled module's `execTransactionFromModule` addressed to the Safe with a `setGuard` call succeeds and moves the guard out of its slot. The guard rejects that call on the signed path (CL130L2-12 / CL141L2-12 / CL150L2-12), so it is not consulted on the module path. LG130L2-3 / LG141L2-3 / LG150L2-3 `moduleCanStillCallOut` ✅ passes: with ConfigLockGuard installed and its `lockedDelay` the Delay Modifier (P4.DelayModifier.8), a plain `execTransactionFromModule` from the Delay Modifier, as an enabled module, to another contract succeeds. • DE130L2-1 / DE141L2-1 / DE150L2-1 also assumes the PauseGuard is not paused. While it is paused, the PauseGuard rejects every `executeNextTx` (`PauseGuard.sol:119`), so the proposal waits until the PauseGuard's admin unpauses it. If `txExpiration` passes first, the proposal expires and has to be proposed again. **On-chain — The setup formally verified above is deployed, and the Delay Modifier's expiration period is 24 hours:** • ⏳ pending deployment. `target()` on the Delay Modifier must return the OwnerlessSafe. • ⏳ pending deployment. `guard()` on the Delay Modifier must return the PauseGuard. • ⏳ pending deployment. The guard slot

#	Statement	Evidence
		(`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:30` / `:42` / `:66` in GnosisSafeL2 v1.3.0 / SafeL2 v1.4.1 / SafeL2 v1.5.0) on the OwnerlessSafe and on the ProposerSafe must hold the ConfigLockGuard. • ⏳ pending deployment. If the OwnerlessSafe runs SafeL2 v1.5.0, its module guard slot (`0xb104e0b93118902c651344349b610029d694cfdec91c589c91ebafbcd0289947`, `ModuleManager.sol:65` in SafeL2 v1.5.0) must hold `address(0)`. The Delay Modifier's call to the OwnerlessSafe is a module transaction, which a v1.5.0 module guard would check (`ModuleManager.sol:94–109`). DE150L2-1 and LG150L2-3 run with it unset. GnosisSafeL2 v1.3.0 and SafeL2 v1.4.1 have no module guard. • ⏳ pending deployment. `paused()` on the PauseGuard must return false. • ⏳ pending deployment. `txExpiration()` on the Delay Modifier must be 86,400 seconds (24 hours). • Assumes the OwnerlessSafe and ProposerSafe run GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.2, P1.9), the versions the DE, LG and CL rules above check. Each of DE130L2-1 / DE141L2-1 / DE150L2-1 runs both Safes on the same version. **FV — The Term multisig holders on the OwnerlessSafe may execute DEVOPS_ROLE methods directly from the OwnerlessSafe:** • DE130L2-2 / DE141L2-2 / DE150L2-2 `ownerlessOwnersExecuteDevopsMethodThroughExecTransaction` ✅ passes. In one execution: ◦ An owner of the OwnerlessSafe (a Term multisig holder, P1.4) approves with `approveHash` an OwnerlessSafe transaction that calls a DEVOPS_ROLE method on a protocol contract. ◦ Anyone submits it through `execTransaction` with that approval as the signature. ◦ The call passes `onlyRole(DEVOPS_ROLE)` because the OwnerlessSafe holds `DEVOPS_ROLE` (P1.1), and the method runs. • DE130L2-2 / DE141L2-2 / DE150L2-2 sets the OwnerlessSafe's threshold to 1. By SE130L2-6 / SE141L2-6 / SE150L2-6 (P1.2), with the real threshold of 9 (P1.4), all nine owners' approvals take the same path. • DE130L2-2 / DE141L2-2 / DE150L2-2 assumes the OwnerlessSafe has no guard. Its ConfigLockGuard lets the call through. Its `lockedDelay` is the Delay Modifier (P4.DelayModifier.8), and a DEVOPS_ROLE method call to a protocol contract is a plain call to neither the OwnerlessSafe nor the Delay Modifier, so it is not a locked call. CL130L2-10 / CL141L2-10 / CL150L2-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the Delay Modifier included, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so DE130L2-2 / DE141L2-2 / DE150L2-2's path is unchanged.

Proposition 2 — Only the Term multisig holders on the DelayOwnerSafe may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier, and 3. the txCooldown constant for the Delay Modifier.

#	Statement	Evidence
P2.1	Only Modules of Delay Modifier and Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but	Same as P1.6.

#	Statement	Evidence
	they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	
P2.2	Veto Delay Modifier transactions (`Delay.setTxNonce(uint256)`) is only callable by Owner of Delay Modifier, and not by Modules of Delay Modifier.	**FV — `setTxNonce`, which vetoes Delay Modifier transactions, is only callable by Owner of Delay Modifier, and not by Modules of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: `setTxNonce` is one of the Delay Modifier's ten `onlyOwner` functions, and each succeeds only when the caller is the owner (Owner of Delay Modifier). • DM-15 `enabledModuleThatIsNotOwnerCannotSetTxNonce` ✅ passes: in particular, an enabled module (Modules of Delay Modifier) that is not the owner cannot call `setTxNonce`; the call always reverts, whatever the nonce. By P1.7, the owner is not a module, so this covers every module. • DM-16 `ownerCanSetTxNonce` ✅ passes: the owner's `setTxNonce` succeeds and sets `txNonce`. So DM-8 and DM-15 do not hold just because `setTxNonce` always reverts.
P2.3	`Owner of Delay Modifier = DelayOwnerSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier.	**FV — The Owner of Delay Modifier is the single address `owner()` returns:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of the Delay Modifier's ten `onlyOwner` functions succeeds only when the caller is the address `owner()` returns, and from the same state no two different callers can both succeed. So only one address at a time can act as the Owner of Delay Modifier. • DM-19 `ownerCanCallEachDelaySetting` ✅ passes: for each of the ten, some call from that address succeeds. So DM-8 is not vacuous. **On-chain — `owner()` on the Delay Modifier returns the DelayOwnerSafe:** • ⏳ pending deployment. `owner()` on the Delay Modifier must return the DelayOwnerSafe.
P2.4	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	**FV — Callers can only interact with the Safe through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setModuleGuard`, `setup`, `simulateAndRevert`, `fallback` and `receive`:** • SE150L2-1 `safeEntryPointsAreAllAccountedFor` ✅ passes: SafeL2 v1.5.0 has no write functions besides `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setModuleGuard`, `setup`, `simulateAndRevert`, `fallback` and `receive` (the Prover checks `fallback` and `receive` as one entry), and any unsupported call lands in `fallback`. These are P1.2's sixteen entry points plus `setModuleGuard`, which Safe added in v1.5.0. Each blob below covers some of these entry points, and together they cover all seventeen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Safe:** • SE150L2-3 `onlyEnabledModulesCanCallModuleExec` ✅ passes: only an enabled module (Modules of Safe) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • SE150L2-4 `enabledModuleCanMakeTheSafeAct` ✅ passes: an enabled module can call them directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. So SE150L2-3 is not vacuous. **FV — `approveHash` is only callable by Owners of Safe:** • SE150L2-8 `onlyOwnersCanApproveHashes` ✅ passes: in any call, an approval is only recorded when the caller is an owner (Owners of Safe) recording its own approval. So a call to `approveHash` from anyone else records nothing. An owner's recorded approval then counts as that owner's signature in `execTransaction`.

#	Statement	Evidence
		FV — `execTransaction` is callable by anyone, but only executes once a threshold of Owners of Safe have approved: • SE150L2-5 `execTransactionRunsOnlyAfterTheSignatureCheck` ✅ passes: whoever the caller is, `execTransaction` only runs once the signature check has passed for that exact transaction, asking for as many signatures as the threshold. The rule places no restriction on the caller. • SE150L2-12 `eachSignatureAcceptsANewApprovingOwner` ✅ passes: each signature the check accepts comes from a different owner (Owners of Safe) who approved the transaction. For an approval made through `approveHash`, SE150L2-8 ensures the owner recorded it itself. So with SE150L2-5, the Safe acts through `execTransaction` only once as many different owners as the threshold have approved. • SE150L2-6 `ownersCanMakeTheSafeAct` ✅ passes: once the signature threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners (Owners of Safe). So SE150L2-5 is not vacuous. **FV — `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler` and `setModuleGuard` are only callable from Modules of Safe or Owners of Safe:** • SE150L2-7 `settingsOnlyChangeWhenTheSafeCallsItself` ✅ passes: in any call, the owners, threshold, modules, guard, module guard and fallbackHandler only change when the caller is the Safe itself. So a call to any of these nine functions from anyone else changes nothing. • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P2.5), a setting only changes through an enabled module's (Modules of Safe) `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. By the `execTransactionFromModule` and `execTransaction` blobs above, those paths are open only to Modules of Safe and to a threshold of Owners of Safe. • SC150L2-1 `moduleCanChangeSettings`, SC150L2-2 `moduleCanChangeSettingsWithReturnData` ✅ pass: for each function above, a module's (Modules of Safe) `execTransactionFromModule`, and likewise its `execTransactionFromModuleReturnData`, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC150L2-3 `ownersCanChangeSettings` ✅ passes: for each function above, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. Together with SC150L2-1 and SC150L2-2, this shows the restriction is not achieved by nothing working. **FV — `setup` always reverts after deploy:** • SE150L2-9 `setupAlwaysRevertsAfterSetup` ✅ passes: once the Safe is set up, `setup` always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. • The DelayOwnerSafe is set up: its threshold is 5 and it has eleven owners (P2.6). **FV — `simulateAndRevert` always reverts:** • SE150L2-11 `simulateAndRevertAlwaysReverts` ✅ passes: anyone can call `simulateAndRevert`, and it does make a delegatecall, but it always reverts afterwards, so none of its effects persist. **FV — `fallback` is callable by anyone but does not change state, assuming the fallbackHandler is `address(0)`:** • SE150L2-21 `noHandlerFallbackChangesNoState` ✅ passes: with no handler set, `fallback` changes no state at all. It writes no storage, makes no call of any kind (call, delegatecall, staticcall or callcode) and creates or destroys no contract, and a `fallback` call that succeeds carries no ETH, so not even the Safe's balance changes. • SE150L2-23 `storageHookFires`, SE150L2-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE150L2-21's checks do detect storage writes and calls. • SE150L2-22 `noHandlerFallbackSucceeds` ✅ passes: with no handler set, some call to `fallback` succeeds, so SE150L2-21 is not vacuous.

#	Statement	Evidence
		FV — `receive` is callable by anyone but does not change state: • SE150L2-25 `receiveChangesNoState` ✅ passes: whatever the handler, a call that runs `receive` changes no state beyond the ETH it is sent. It writes no storage, makes no call of any kind and creates or destroys no contract. • Here, state means the Safe's storage, which holds its owners, threshold, modules, guard, module guard, fallbackHandler, nonce and approvals. The ETH sent is added to the Safe's balance, which is not part of that storage. • SE150L2-23 `storageHookFires`, SE150L2-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE150L2-25's checks do detect storage writes and calls. • SE150L2-26 `receiveRuns` ✅ passes: some call to the Safe runs `receive` and succeeds, so SE150L2-25 is not vacuous. **On-chain — The DelayOwnerSafe runs SafeL2 v1.5.0, the code the SE150L2 and SC150L2 rules check:** • ⏳ pending deployment. The DelayOwnerSafe must run SafeL2 v1.5.0: `VERSION()` must return `1.5.0`, and storage slot 0 (the proxy's `singleton`) must hold the canonical SafeL2 v1.5.0 singleton. `VERSION()` alone does not tell SafeL2 v1.5.0 from Safe v1.5.0; the singleton does. • So the rules above, which check SafeL2 v1.5.0, hold for the DelayOwnerSafe. They are P1.2's rules carried over to v1.5.0, with `setModuleGuard` added to the settings functions and the module guard added to the settings they track. • Assumes the DelayOwnerSafe runs SafeL2 v1.5.0, the version SE150L2 and SC150L2 check.
P2.5	DelayOwnerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier.	**On-chain — DelayOwnerSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:13` in SafeL2 v1.5.0) on the DelayOwnerSafe must hold `address(0)`. • Assumes the DelayOwnerSafe runs SafeL2 v1.5.0 (`FallbackManager.sol:13`).
P2.6	`Owners of DelayOwnerSafe = Term multisig holders on DelayOwnerSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier.	**On-chain — Owners of DelayOwnerSafe are eleven Term multisig holders, and five of them must approve each transaction:** • ⏳ pending deployment. The DelayOwnerSafe must run SafeL2 v1.5.0 (`VERSION()` and storage slot 0, as in P2.4), with threshold 5 (`getThreshold`) and eleven owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the DelayOwnerSafe runs SafeL2 v1.5.0. • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P2.7	`Modules of DelayOwnerSafe = null`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier.	**On-chain — The Modules of DelayOwnerSafe list is empty:** • ⏳ pending deployment. `getModulesPaginated` on the DelayOwnerSafe must return an empty list. • Assumes the DelayOwnerSafe runs SafeL2 v1.5.0.
P2.8	The Term multisig holders on the DelayOwnerSafe may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot	**FV — A Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued:** • DP-31 `onlyExecuteNextTxReachesTheTarget` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, only `executeNextTx` hands a transaction to the Delay Modifier's target. So `executeNextTx` is the only way a queued transaction is executed. • DP-25 `executeNextTxConsumesOnlyTheEntryAtTxNonce` ✅ passes: `executeNextTx` only executes the queue entry at

#	Statement	Evidence
	be executed until txCooldown has passed since it was queued.	`txNonce`, and moves `txNonce` forward by one. • DP-27 `executeNextTxRevertsDuringCooldown` ✅ passes: until `txCooldown` has passed since the entry at `txNonce` was queued, `executeNextTx` always reverts, whoever calls it and whatever it is given. With DP-25, no queue entry can be executed before its own cooldown has passed. This is the veto window. • DP-26 `queuedTransactionExecutesAfterCooldown` ✅ passes: once the cooldown has passed, before the entry expires and with the PauseGuard not paused, anyone's `executeNextTx` executes it. So DP-27 is not vacuous, and the veto window ends when the cooldown does. • `executeNextTx` compares against `txCooldown` as it is when `executeNextTx` runs, not as it was when the transaction was queued (`Delay.sol:208–211`). So the veto window lasts `txCooldown` only while `txCooldown` is held constant, as Proposition 2 holds it. `txExpiration` only limits how long after the cooldown a transaction can still be executed (`Delay.sol:212–218`), so it does not affect the veto window. **FV — Once a veto moves `txNonce` past a transaction, that transaction can never be executed:** • DP-29 `vetoedTransactionCannotExecute` ✅ passes: a module queues a transaction, the owner's `setTxNonce` moves `txNonce` past it, and `executeNextTx` with that transaction then reverts. • DP-30 `entryPassedByTxNonceNeverExecutes` ✅ passes: from any state, once `txNonce` is past a queue entry, `executeNextTx` with that entry's transaction reverts, unless the same transaction has been queued again and sits at `txNonce`. Queuing it again makes a new entry, with its own veto window. • DP-24 `txNonceNeverDecreases` ✅ passes: no write function moves `txNonce` backwards, so a vetoed entry stays vetoed, whatever later happens to `txCooldown` or `txExpiration`. **On-chain — The veto window is 24 hours:** • ⏳ pending deployment. `txCooldown()` on the Delay Modifier must be 86,400 seconds (24 hours). **FV — The Term multisig holders on the DelayOwnerSafe can veto Delay Modifier transactions within the 24-hour veto window, directly from the DelayOwnerSafe:** • DV150L2-1 `approvedOwnersVetoThroughExecTransaction` ✅ passes. In one execution: ◦ An owner of the DelayOwnerSafe (a Term multisig holder, P2.6) approves a `setTxNonce(n)` transaction with `approveHash`, for an `n` the Delay Modifier accepts. ◦ `execTransaction` runs it with that approval as the signature. ◦ With the DelayOwnerSafe as the Delay Modifier's owner (P2.3), the call succeeds and sets the Delay Modifier's `txNonce` to `n`. • DV150L2-1 sets the DelayOwnerSafe's threshold to 1. By SE150L2-6 (P2.4), with the real threshold of 5 (P2.6), five owners' approvals take the same path. • DV150L2-1 assumes the DelayOwnerSafe owns the Delay Modifier (P2.3) and has no guard. The DelayOwnerSafe's guard is ConfigLockGuard (read below), which the next two bullets cover. The holders can veto at any point in the 24-hour veto window: by DP-32 (below), the owner's `setTxNonce` succeeds whatever the time. • ConfigLockGuard lets the veto through. CL150L2-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. The veto, `setTxNonce(n)` on the Delay Modifier, is a plain call. It is not one of the nine calls the guard locks on its locked modifier (CL150L2-6 to CL150L2-9, CL150L2-24). The guard's `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so DV150L2-1's path is unchanged. • OD150L2-4 `guardedSafeCanStillVeto` ✅ passes. In this rule, ConfigLockGuard is installed on the DelayOwnerSafe with the Delay Modifier as its locked modifier, and the DelayOwnerSafe owns the Delay Modifier. An `execTransaction` to the Delay Modifier of `setTxNonce(n)` succeeds and `txNonce` becomes `n`, for an `n` the Delay Modifier accepts. • OD150L2-4 stubs the signature check. By P2.4, `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150L2-5, SE150L2-12), and their approvals pass it (SE150L2-6).

#	Statement	Evidence
		• DP-32 `ownerCanSetTxNonceAtAnyTime` ✅ passes: whatever the time, `txCooldown`, `txExpiration` and pause state, the Delay Modifier owner's `setTxNonce(n)` succeeds and sets `txNonce` to `n`, for any `n` the Delay Modifier accepts. **On-chain — The setup formally verified above is deployed:** • ⏳ pending deployment. The DelayOwnerSafe's guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in SafeL2 v1.5.0) must hold ConfigLockGuard. This is the same requirement as P4.DelayOwnerSafe.5. • ⏳ pending deployment. `lockedDelay()` on ConfigLockGuard must return the Delay Modifier. • Assumes the DelayOwnerSafe runs SafeL2 v1.5.0, the version CL150L2, DV150L2-1 and OD150L2-4 check (`GuardManager.sol:66`).

Proposition 3 — Only the Term multisig holders on the PauseSafe may pause, and only the Term multisig holders on the ProposerSafe may unpause, the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.

#	Statement	Evidence
P3.1	Only the PauseGuard pauser and the PauseGuard admin can interact with the PauseGuard, except `checkTransaction` and `checkAfterExecution`, which anyone can call but which do not change state.	**FV — Callers can only change the PauseGuard through `pause`, `unpause` and `setPauser`:** • DP-9 `pausedOnlyChangesThroughPauseOrUnpause` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, `paused` changes only through `pause` or `unpause`. • DP-13 `pauserOnlyChangesThroughSetPauser` ✅ passes: over every write function of both contracts, the pauser only changes through `setPauser`. • DP-12 `pauseGuardWriteFunctionsAreTheKnownThree` ✅ passes: PauseGuard has no fallback, and its write functions are exactly `pause`, `unpause` and `setPauser`. • `checkTransaction` and `checkAfterExecution` are `view` functions (`PauseGuard.sol:117`, `PauseGuard.sol:123`), so they change no state. **FV — `pause` is only callable by the PauseGuard pauser, and `unpause` and `setPauser` only by the PauseGuard admin:** • DP-3 `pauseRevertsForAnyoneButThePauser` ✅ passes: anyone who is not the pauser is rejected. • DP-6 `unpauseRevertsForAnyoneButTheAdmin` ✅ passes: anyone who is not the admin is rejected. • DP-10 `setPauserRevertsForAnyoneButTheAdmin` ✅ passes: anyone who is not the admin is rejected.

Pause

P3.Pause: Only the Term multisig holders on the PauseSafe may pause the execution of Delay Modifier transactions.

#	Statement	Evidence
P3.Pause.1	Pause Delay Modifier executions (`PauseGuard.pause()`) is only callable by the PauseGuard pauser, and not by the PauseGuard admin.	**FV — `pause` is only callable by the PauseGuard pauser, and not by the PauseGuard admin:** • DP-3 `pauseRevertsForAnyoneButThePauser` ✅ passes: anyone who is not the pauser is rejected. • DP-4 `adminAloneCannotPause` ✅ passes: being the admin does not let a caller pause. • DP-5 `pauseSucceedsForThePauser` ✅ passes: from an unpaused state, the pauser's `pause()` succeeds and `paused` is then true. So DP-3 and DP-4 do not hold just because `pause` always reverts.
P3.Pause.2	`PauseGuard pauser = PauseSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — The PauseGuard's pauser is the PauseSafe:** • ⏳ pending deployment. `pauser()` on the PauseGuard must return the PauseSafe.
P3.Pause.3	Only Modules of PauseSafe and Owners of PauseSafe can interact with the PauseSafe, except `execTransaction`, which anyone can call only if Owners of PauseSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4. The PauseSafe runs SafeL2 v1.5.0, like the DelayOwnerSafe (P3.Pause.5).
P3.Pause.4	PauseSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — PauseSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:13` in SafeL2 v1.5.0) on the PauseSafe must hold `address(0)`. • Assumes the PauseSafe runs SafeL2 v1.5.0 (`FallbackManager.sol:13`).
P3.Pause.5	`Owners of PauseSafe = Term multisig holders on PauseSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — Owners of PauseSafe are nine Term multisig holders, and two of them must approve each transaction:** • ⏳ pending deployment. The PauseSafe must run SafeL2 v1.5.0 (`VERSION()` and storage slot 0, as in P2.4), with threshold 2 (`getThreshold`) and nine owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the PauseSafe runs SafeL2 v1.5.0.
P3.Pause.6	`Modules of PauseSafe = null`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and	**On-chain — The Modules of PauseSafe list is empty:** • ⏳ pending deployment. `getModulesPaginated` on the PauseSafe must return an empty list. • Assumes the PauseSafe runs SafeL2 v1.5.0.

#	Statement	Evidence
	ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	
P3.Pause.7	The Term multisig holders on the PauseSafe may pause the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed.	**FV — The PauseSafe's owners can have it pause the PauseGuard, and a pause stops every Delay Modifier execution:** • DP-5 `pauseSucceedsForThePauser` ✅ passes: from an unpaused state, the pauser's `pause()` succeeds and `paused` is then true. • SE150L2-6 `ownersCanMakeTheSafeAct` ✅ passes: once the signature threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the PauseSafe's owners (P3.Pause.5) can have it call `pause` on the PauseGuard. • DP-15 `pausedBlocksExecuteNextTx` ✅ passes: with the PauseGuard installed and paused, `executeNextTx` reverts and nothing reaches the Delay Modifier's target, for any transaction arguments and any queue state. • DP-16 `checkTransactionRevertsWhilePaused` ✅ passes: called directly, a paused PauseGuard rejects every transaction from every caller. • DP-31 `onlyExecuteNextTxReachesTheTarget` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, only `executeNextTx` hands a transaction to the Delay Modifier's target. With DP-15, a pause stops every transaction the Delay Modifier can send. **FV — If a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed:** • DP-15 `pausedBlocksExecuteNextTx` ✅ passes (above): while the PauseGuard is paused, `executeNextTx` reverts, so the transaction cannot be executed during the pause. • DP-28 `executeNextTxRevertsAfterExpiration` ✅ passes: with a non-zero `txExpiration`, once more than `txCooldown` plus `txExpiration` has passed since the head entry was queued, `executeNextTx` with it reverts. That stays true at every later time, so a transaction whose window ended during the pause cannot be executed after the unpause either. • DM-12 `anyoneCanSkipExpired` ✅ passes: whether a call to `executeNextTx` or `skipExpired` succeeds, and what it changes, does not depend on who the caller is. DM-18 `outsiderCanSkipExpired` ✅ passes: a caller that is neither the owner nor a module can skip a queue entry. • DP-33 `skipExpiredSucceedsWhilePaused` ✅ passes: with the PauseGuard installed and paused, once the head entry has expired, anyone's `skipExpired` succeeds, moves `txNonce` past it and hands nothing to the Delay Modifier's target. So a transaction whose window ended during the pause can be skipped during the pause. • The Owner of Delay Modifier can also move `txNonce` past it with a veto, `setTxNonce` (P2.2). • DP-30 `entryPassedByTxNonceNeverExecutes` ✅ passes: once `txNonce` is past the entry, `executeNextTx` with that entry's transaction reverts, unless the same transaction has been queued again and sits at `txNonce`. Queuing it again makes a new entry, with its own veto window. **On-chain — The PauseGuard is the Delay Modifier's guard, and it is not paused:** • ⏳ pending deployment. `guard()` on the Delay Modifier must return the PauseGuard, as in P1.13. • ⏳ pending deployment. `paused()` on the PauseGuard must return false. **On-chain — The Delay Modifier's expiration period is 24 hours:** • ⏳ pending deployment. `txExpiration()` on the Delay Modifier must be 86,400 seconds (24 hours), as in P1.13.

Admin

P3.Admin: Only the Term multisig holders on the ProposerSafe may unpause the execution of Delay Modifier transactions.

#	Statement	Evidence
P3.Admin.1	Unpause Delay Modifier executions (`PauseGuard.unpause()`) is only callable by the PauseGuard admin, and not by the PauseGuard pauser.	**FV — `unpause` is only callable by the PauseGuard admin, and not by the PauseGuard pauser:** • DP-6 `unpauseRevertsForAnyoneButTheAdmin` ✅ passes: anyone who is not the admin is rejected. • DP-7 `pauserAloneCannotUnpause` ✅ passes: being the pauser, without being the admin, does not let a caller lift a pause. • DP-8 `unpauseSucceedsForTheAdmin` ✅ passes: from a paused state, the admin's `unpause()` succeeds and `paused` is then false. So DP-6 and DP-7 do not hold just because `unpause` always reverts.
P3.Admin.2	`PauseGuard admin = ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — The ProposerSafe is the PauseGuard's admin:** • ⏳ pending deployment. `admin()` on the PauseGuard must return the ProposerSafe. `admin` is `immutable` (`PauseGuard.sol:17`), so the PauseGuard's constructor sets it.
P3.Admin.3	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.9.
P3.Admin.4	ProposerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	Same as P1.10.
P3.Admin.5	`Owners of ProposerSafe = Term multisig holders on ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	Same as P1.11.

#	Statement	Evidence
P3.Admin.6	`Modules of ProposerSafe = null`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	Same as P1.12.
P3.Admin.7	The Term multisig holders on the ProposerSafe may unpause the execution of Delay Modifier transactions.	**FV — The ProposerSafe's owners can have it unpause the PauseGuard, and unpausing reopens Delay Modifier execution:** • DP-8 `unpauseSucceedsForTheAdmin` ✅ passes: from a paused state, the admin's `unpause()` succeeds and `paused` is then false. • SE130L2-6 / SE141L2-6 / SE150L2-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the ProposerSafe's owners (P3.Admin.5) can have it call `unpause` on the PauseGuard. • The ProposerSafe's guard is ConfigLockGuard (P4.ProposerSafe.5). Its call to `unpause` on the PauseGuard is a plain call to neither the ProposerSafe nor its locked Delay Modifier, so it is not a locked call. CL130L2-10 / CL141L2-10 / CL150L2-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so the call is unchanged. • DP-17 `unpauseReopensExecuteNextTx` ✅ passes: with the PauseGuard installed and paused, after the admin unpauses, a following `executeNextTx` can succeed and reach the Delay Modifier's target. • DP-18 `checkTransactionAcceptsWhileNotPaused` ✅ passes: while not paused, the PauseGuard accepts any transaction from any caller, so it adds no revert of its own to `executeNextTx`. **On-chain — The PauseGuard is the Delay Modifier's guard:** • ⏳ pending deployment. `guard()` on the Delay Modifier must return the PauseGuard, as in P1.13.

Proposition 4 — The owners for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe. The fallbackHandler, modules, guard, module guard for the OwnerlessSafe can be changed only by the Term multisig holders on the ProposerSafe. The following: 1. the fallbackHandler, modules, guard for the ProposerSafe and DelayOwnerSafe, and 2. the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by anyone.

ProposerSafe

P4.ProposerSafe: The owners for the ProposerSafe can only be changed by Term multisig holders on the ProposerSafe; the fallbackHandler, modules, guard for the ProposerSafe cannot be changed by anyone.

#	Statement	Evidence
P4.ProposerSafe.1	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.9.
P4.ProposerSafe.2	ProposerSafe fallbackHandler is `address(0)`, which can only be changed by Owners of ProposerSafe and Modules of ProposerSafe.	**On-chain — ProposerSafe fallbackHandler is `address(0)`:** • Same as P1.10. **FV — ProposerSafe fallbackHandler can only be changed by Owners of ProposerSafe or Modules of ProposerSafe:** • SC130L2-4 / SC141L2-4 / SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included (on SafeL2 v1.5.0, while no fallbackHandler is set, P1.10), the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE130L2-3 / SE141L2-3 / SE150L2-3), and `execTransaction` only acts once as many Owners of ProposerSafe as the threshold have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12). So only Modules of ProposerSafe or Owners of ProposerSafe can change the fallbackHandler.
P4.ProposerSafe.3	`Modules of ProposerSafe = null`, which can only be changed by Owners of ProposerSafe because `Modules of ProposerSafe = null`.	**On-chain — The Modules of ProposerSafe list is empty:** • Same as P1.12. **FV — Modules of ProposerSafe can only be changed by Owners of ProposerSafe because `Modules of ProposerSafe = null`:** • SC130L2-4 / SC141L2-4 / SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included (on SafeL2 v1.5.0, while no fallbackHandler is set, P1.10), the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE130L2-3 / SE141L2-3 / SE150L2-3), and `execTransaction` only acts once as many Owners of ProposerSafe as the threshold have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12). • The ProposerSafe has no module (P1.12), so only the Owners of ProposerSafe can change the module list.
P4.ProposerSafe.4	`Owners of ProposerSafe = Term multisig holders on the ProposerSafe`, which can only be changed by Term multisig holders on the	**On-chain — Owners of ProposerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P1.11.

#	Statement	Evidence
	ProposerSafe because `Modules of ProposerSafe = null`.	**FV — Owners of ProposerSafe can only be changed by Term multisig holders on the ProposerSafe because `Modules of ProposerSafe = null`:** • SC130L2-4 / SC141L2-4 / SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included (on SafeL2 v1.5.0, while no fallbackHandler is set, P1.10), the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE130L2-3 / SE141L2-3 / SE150L2-3), and `execTransaction` only acts once five Owners of ProposerSafe, the Term multisig holders (P1.11), have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12). • The ProposerSafe has no module (P1.12), so only the Term multisig holders on the ProposerSafe can change the owners and threshold.
P4.ProposerSafe.5	The owners for the ProposerSafe can be changed by Term multisig holders on the ProposerSafe; the fallbackHandler, modules, guard for the ProposerSafe cannot be changed by Term multisig holders on the ProposerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the ProposerSafe can change Owners of ProposerSafe through `execTransaction`:** • CL130L2-21 / CL141L2-21 / CL150L2-21 `guardedOwnersCanStillChangeOtherSettings` ✅ passes, for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL130L2-21 / CL141L2-21 / CL150L2-21 stubs the signature check. By P1.9, `execTransaction` only acts once five Owners of ProposerSafe, the Term multisig holders (P1.11), have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12), and their approvals pass it (SE130L2-6 / SE141L2-6 / SE150L2-6). **FV — The Term multisig holders on the ProposerSafe cannot change ProposerSafe fallbackHandler, Modules of ProposerSafe or ProposerSafe guard through `execTransaction` if ProposerSafe guard is ConfigLockGuard, so nobody can change them:** • CL130L2-2 / CL141L2-2 / CL150L2-2 `checkTransactionRejectsSelfSetGuard`, CL130L2-3 / CL141L2-3 / CL150L2-3 `checkTransactionRejectsSelfEnableModule`, CL130L2-4 / CL141L2-4 / CL150L2-4 `checkTransactionRejectsSelfDisableModule`, CL130L2-5 / CL141L2-5 / CL150L2-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL130L2-12 / CL141L2-12 / CL150L2-12 `guardedSafeCannotCallSetGuard`, CL130L2-13 / CL141L2-13 / CL150L2-13 `guardedSafeCannotCallEnableModule`, CL130L2-14 / CL141L2-14 / CL150L2-14 `guardedSafeCannotCallDisableModule`, CL130L2-15 / CL141L2-15 / CL150L2-15 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL130L2-17 / CL141L2-17 / CL150L2-17 `guardedSelfCallNeverChangesGuard`, CL130L2-18 / CL141L2-18 / CL150L2-18 `guardedSelfCallNeverChangesModules`, CL130L2-19 / CL141L2-19 / CL150L2-19 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of ProposerSafe list and the fallbackHandler unchanged. • CL130L2-1 / CL141L2-1 / CL150L2-1 `checkTransactionRejectsDelegateCall`, CL130L2-16 / CL141L2-16 / CL150L2-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of ProposerSafe list or the fallbackHandler directly. • By P1.9, these settings only change when the caller is the Safe itself (SE130L2-7 / SE141L2-7 / SE150L2-7), so an `execTransaction` to any other address cannot change them either. • These rules leave `lockedDelay` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P4.ProposerSafe.2 and P4.ProposerSafe.3, only Owners of ProposerSafe or Modules of ProposerSafe can change the

#	Statement	Evidence
		fallbackHandler and the Modules of ProposerSafe list, and SC130L2-4 / SC141L2-4 / SC150L2-4 (P4.ProposerSafe.2) covers the guard the same way. The ProposerSafe has no module (P4.ProposerSafe.3), so nobody can change them. **On-chain — ProposerSafe guard is ConfigLockGuard:** • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:30` / `:42` / `:66` in GnosisSafeL2 v1.3.0 / SafeL2 v1.4.1 / SafeL2 v1.5.0) on the ProposerSafe must hold ConfigLockGuard. • Assumes the ProposerSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.9), the versions the CL rules above check.

DelayOwnerSafe

P4.DelayOwnerSafe: The owners for the DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe; the fallbackHandler, modules, guard for the DelayOwnerSafe cannot be changed by anyone.

#	Statement	Evidence
P4.DelayOwnerSafe.1	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4.
P4.DelayOwnerSafe.2	DelayOwnerSafe fallbackHandler is `address(0)`, which can only be changed by Owners of DelayOwnerSafe and Modules of DelayOwnerSafe.	**On-chain — DelayOwnerSafe fallbackHandler is `address(0)`:** • Same as P2.5. **FV — DelayOwnerSafe fallbackHandler can only be changed by Owners of DelayOwnerSafe or Modules of DelayOwnerSafe:** • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set, the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150L2-3), and `execTransaction` only acts once as many Owners of DelayOwnerSafe as the threshold have approved (SE150L2-5, SE150L2-12). So only Modules of DelayOwnerSafe or Owners of DelayOwnerSafe can change the fallbackHandler.
P4.DelayOwnerSafe.3	`Owners of DelayOwnerSafe = Term multisig holders on the DelayOwnerSafe`, which can only be changed by Term multisig holders on the	**On-chain — Owners of DelayOwnerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P2.6. **FV — Owners of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe or Modules**

#	Statement	Evidence
	DelayOwnerSafe and Modules of DelayOwnerSafe.	**of DelayOwnerSafe:** • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set, the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150L2-3), and `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150L2-5, SE150L2-12). So only Modules of DelayOwnerSafe or the Term multisig holders on the DelayOwnerSafe can change the owners and threshold.
P4.DelayOwnerSafe.4	`Modules of DelayOwnerSafe = null`, which can only be changed by Owners of DelayOwnerSafe because `Modules of DelayOwnerSafe = null`.	**On-chain — The Modules of DelayOwnerSafe list is empty:** • Same as P2.7. **FV — Modules of DelayOwnerSafe can only be changed by Owners of DelayOwnerSafe because `Modules of DelayOwnerSafe = null`:** • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set, the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150L2-3), and `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150L2-5, SE150L2-12). • The DelayOwnerSafe has no module (P2.7), so only the Owners of DelayOwnerSafe can change the module list.
P4.DelayOwnerSafe.5	The owners for the DelayOwnerSafe can be changed by Term multisig holders on the DelayOwnerSafe; the fallbackHandler, modules, guard for the DelayOwnerSafe cannot be changed by Term multisig holders on the DelayOwnerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the DelayOwnerSafe can change Owners of DelayOwnerSafe through `execTransaction`:** • CL150L2-21 `guardedOwnersCanStillChangeOtherSettings` ✅ passes on SafeL2 v1.5.0, the DelayOwnerSafe's code (P2.4), for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL150L2-21 stubs the signature check. By P2.4, `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150L2-5, SE150L2-12), and their approvals pass it (SE150L2-6). **FV — The Term multisig holders on the DelayOwnerSafe cannot change DelayOwnerSafe fallbackHandler, Modules of DelayOwnerSafe or DelayOwnerSafe guard through `execTransaction` if DelayOwnerSafe guard is ConfigLockGuard, so nobody can change them:** • CL150L2-2 `checkTransactionRejectsSelfSetGuard`, CL150L2-3 `checkTransactionRejectsSelfEnableModule`, CL150L2-4 `checkTransactionRejectsSelfDisableModule`, CL150L2-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL150L2-12 `guardedSafeCannotCallSetGuard`, CL150L2-13 `guardedSafeCannotCallEnableModule`, CL150L2-14 `guardedSafeCannotCallDisableModule`, CL150L2-15 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL150L2-17 `guardedSelfCallNeverChangesGuard`, CL150L2-18 `guardedSelfCallNeverChangesModules`, CL150L2-19 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of DelayOwnerSafe list and the fallbackHandler unchanged. • CL150L2-1 `checkTransactionRejectsDelegateCall`, CL150L2-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of DelayOwnerSafe list or the

#	Statement	Evidence
		fallbackHandler directly. • By P2.4, these settings only change when the caller is the Safe itself (SE150L2-7), so an `execTransaction` to any other address cannot change them either. • These rules run on SafeL2 v1.5.0, the DelayOwnerSafe's code (P2.4), and leave `lockedDelay` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P4.DelayOwnerSafe.2 and P4.DelayOwnerSafe.4, only Owners of DelayOwnerSafe or Modules of DelayOwnerSafe can change the fallbackHandler and the Modules of DelayOwnerSafe list, and SC150L2-4 (P4.DelayOwnerSafe.2) covers the guard the same way. The DelayOwnerSafe has no module (P4.DelayOwnerSafe.4), so nobody can change them. **On-chain — DelayOwnerSafe guard is ConfigLockGuard:** • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in SafeL2 v1.5.0) on the DelayOwnerSafe must hold ConfigLockGuard. • Assumes the DelayOwnerSafe runs SafeL2 v1.5.0, the version CL150L2 checks (`GuardManager.sol:66`).

Delay Modifier

P4.DelayModifier: The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by anyone.

#	Statement	Evidence
P4.DelayModifier.1	Only Modules of Delay Modifier or Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	Same as P1.6.
P4.DelayModifier.2	The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can only be changed by Owner of Delay Modifier, not by Modules of Delay Modifier.	**FV — The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can only be changed by Owner of Delay Modifier, not Modules of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of the Delay Modifier's ten `onlyOwner` functions, which include `setTxCooldown`, `setTxExpiration`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership`, succeeds only when the caller is the owner (Owner of Delay Modifier). • DM-6 `ownerOnlyChangesThroughOwnableTransfer`, DM-2 `modulesOnlyChangeThroughOwnerEnableOrDisable`, DM-7 `guardOnlyChangesThroughOwnerSetGuard` ✅ pass: over every write function, the owner, the module list and the guard change only when the caller is the owner. • By P1.7, the owner is not a module, so no Module of Delay Modifier can change these settings.
P4.DelayModifier.3	`Owner of Delay Modifier = DelayOwnerSafe`, which can only be changed by the DelayOwnerSafe.	**On-chain — Owner is DelayOwnerSafe:** • Same as P2.3. **FV — Owner of Delay Modifier can only be changed by Owner of Delay Modifier:** • DM-6 `ownerOnlyChangesThroughOwnableTransfer` ✅ passes: over every write function, the owner changes only through `transferOwnership` or `renounceOwnership`, and only when the caller is the current owner (Owner of Delay

#	Statement	Evidence
		Modifier). • So, since the owner is the DelayOwnerSafe, only the DelayOwnerSafe can change it.
P4.DelayModifier.4	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4.
P4.DelayModifier.5	DelayOwnerSafe fallbackHandler is `address(0)`, which cannot be changed by anyone, P4.DelayOwnerSafe.	**Already proven — DelayOwnerSafe fallbackHandler is `address(0)`:** • Same as P2.5. **Already proven — DelayOwnerSafe fallbackHandler cannot be changed by anyone:** • See P4.DelayOwnerSafe: by P4.DelayOwnerSafe.5, the Term multisig holders on the DelayOwnerSafe cannot change the fallbackHandler, so nobody can.
P4.DelayModifier.6	`Owners of DelayOwnerSafe = Term multisig holders on the DelayOwnerSafe`, which can only be changed by Term multisig holders on the DelayOwnerSafe, P4.DelayOwnerSafe.	**Already proven — Owners of DelayOwnerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P2.6. **Already proven — Owners of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe:** • See P4.DelayOwnerSafe.
P4.DelayModifier.7	`Modules of DelayOwnerSafe = null`, which cannot be changed by anyone, P4.DelayOwnerSafe.	**Already proven — The Modules of DelayOwnerSafe list is empty:** • Same as P2.7. **Already proven — Modules of DelayOwnerSafe cannot be changed by anyone:** • See P4.DelayOwnerSafe: by P4.DelayOwnerSafe.5, the Term multisig holders on the DelayOwnerSafe cannot change the Modules of DelayOwnerSafe list, so nobody can.
P4.DelayModifier.8	The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by Term multisig holders on the DelayOwnerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the DelayOwnerSafe cannot change the owner, target, modules, guard, txCooldown or txExpiration of the Delay Modifier through `execTransaction` if DelayOwnerSafe guard is ConfigLockGuard, so nobody can change them:** • CL150L2-6 `checkTransactionRejectsModifierOwnershipChange`, CL150L2-7 `checkTransactionRejectsModifierModuleChange`, CL150L2-8 `checkTransactionRejectsModifierSetGuard`, CL150L2-9 `checkTransactionRejectsModifierSetAvatarOrTarget` ✅ pass: called by the Safe, the guard reverts on any transaction to its locked modifier carrying `transferOwnership`, `renounceOwnership`, `enableModule`, `disableModule`, `setGuard`, `setAvatar` or `setTarget`. • CL150L2-24 `checkTransactionRejectsModifierTimingChange` ✅ passes: called by the Safe, the guard reverts on any transaction to its locked modifier carrying `setTxCooldown` or `setTxExpiration`. • CL150L2-23 `guardedSafeCannotCallLockedModifierFunctions` ✅ passes: with ConfigLockGuard installed, an `execTransaction` addressed to the locked modifier with any of these calls reverts, whatever the arguments, value and operation.

#	Statement	Evidence
		• CL150L2-1 `checkTransactionRejectsDelegateCall`, CL150L2-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so every `execTransaction` to the Delay Modifier that succeeds is a plain call carrying some other selector. • OD150L2-1 `unlockedDelayCallsNeverChangeOwnerGuardAvatarOrTarget`, OD150L2-2 `unlockedDelayCallsNeverChangeModules` ✅ pass: over every Delay Modifier write function except those nine and `setUp`, and for any caller, the owner, guard, target and module list do not change. • OD150L2-5 `unlockedDelayCallsNeverChangeTxCooldownOrTxExpiration` ✅ passes: over the same functions and any caller, txCooldown and txExpiration do not change. • An `execTransaction` to any other address reaches the Delay Modifier, if at all, with that address as the caller, not the owner, so it cannot change these settings (P4.DelayModifier.2). • The CL150L2 rules leave `lockedDelay` unconstrained, so they lock whichever modifier ConfigLockGuard was built with. The deployed guard's locked modifier is the Delay Modifier (below). • By P4.DelayModifier.2 and P4.DelayModifier.3, only the DelayOwnerSafe can change the owner, target, modules, guard, txCooldown and txExpiration. The DelayOwnerSafe acts only through `execTransaction` or its modules (P4.DelayModifier.4, P4.DelayModifier.5). It has no module (P4.DelayModifier.7), so nobody can change them. **On-chain — DelayOwnerSafe guard is ConfigLockGuard, with the Delay Modifier as its locked modifier:** • Same as P4.DelayOwnerSafe.5: the DelayOwnerSafe's guard slot must hold ConfigLockGuard. • ⏳ pending deployment. `lockedDelay()` on ConfigLockGuard must return the Delay Modifier. • Assumes the DelayOwnerSafe runs SafeL2 v1.5.0, the version CL150L2 and OD150L2 check.

OwnerlessSafe

P4.OwnerlessSafe: The owners for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe and Term multisig holders on the OwnerlessSafe; the fallbackHandler, modules, guard, module guard for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe.

#	Statement	Evidence
P4.OwnerlessSafe.1	Only Modules of OwnerlessSafe and Owners of OwnerlessSafe can interact with the OwnerlessSafe, except `execTransaction`, which anyone can call only if Owners of OwnerlessSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.2.
P4.OwnerlessSafe.2	OwnerlessSafe fallbackHandler is `address(0)`, which can only be changed by Modules of OwnerlessSafe and Owners of OwnerlessSafe.	**On-chain — OwnerlessSafe fallbackHandler is `address(0)`:** • Same as P1.3. **FV — OwnerlessSafe fallbackHandler can only be changed by Owners of OwnerlessSafe or Modules of OwnerlessSafe:**

#	Statement	Evidence
		• SC130L2-4 / SC141L2-4 / SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included (on SafeL2 v1.5.0, while no fallbackHandler is set, P1.3), the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE130L2-3 / SE141L2-3 / SE150L2-3), and `execTransaction` only acts once as many Owners of OwnerlessSafe as the threshold have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12). So only Modules of OwnerlessSafe or Owners of OwnerlessSafe can change the fallbackHandler.
P4.OwnerlessSafe.3	`Owners of OwnerlessSafe = Term multisig holders on the OwnerlessSafe`, which can only be changed by Term multisig holders on the OwnerlessSafe and Modules of OwnerlessSafe.	**On-chain — Owners of OwnerlessSafe are these nine Term multisig holders, and all nine must approve each transaction:** • Same as P1.4. **FV — Owners of OwnerlessSafe can only be changed by Term multisig holders on the OwnerlessSafe or Modules of OwnerlessSafe:** • SC130L2-4 / SC141L2-4 / SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included (on SafeL2 v1.5.0, while no fallbackHandler is set, P1.3), the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE130L2-3 / SE141L2-3 / SE150L2-3), and `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12). So only Modules of OwnerlessSafe or the Term multisig holders on the OwnerlessSafe can change the owners and threshold.
P4.OwnerlessSafe.4	`Modules of OwnerlessSafe = Delay Modifier`, which can only be changed by Term multisig holders on the OwnerlessSafe and Delay Modifier.	**On-chain — The Modules of OwnerlessSafe list contains only the Delay Modifier:** • Same as P1.5. **FV — Modules of OwnerlessSafe can only be changed by Term multisig holders on the OwnerlessSafe or Delay Modifier:** • SC130L2-4 / SC141L2-4 / SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included (on SafeL2 v1.5.0, while no fallbackHandler is set, P1.3), the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE130L2-3 / SE141L2-3 / SE150L2-3), and `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12). • The OwnerlessSafe's only module is the Delay Modifier (P1.5), so only the Delay Modifier or the Term multisig holders on the OwnerlessSafe can change the module list.
P4.OwnerlessSafe.5	Only Modules of Delay Modifier or Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions	Same as P1.6.

#	Statement	Evidence
	queued by Modules of Delay Modifier or Owner of Delay Modifier.	
P4.OwnerlessSafe.6	Only Modules of Delay Modifier can queue transactions to Delay Modifier. Owner of Delay Modifier cannot queue transactions to Delay Modifier.	Same as P1.7.
P4.OwnerlessSafe.7	`Modules of Delay Modifier = ProposerSafe`, which cannot be changed by anyone, P4.DelayModifier.	**On-chain — The Modules of Delay Modifier list contains only the ProposerSafe:** • Same as P1.8. **FV — Modules of Delay Modifier can only be changed by Owner of Delay Modifier:** • DM-2 `modulesOnlyChangeThroughOwnerEnableOrDisable` ✅ passes: over every write function, the module list changes only when the caller is the owner (Owner of Delay Modifier). **Already proven — Modules of Delay Modifier cannot be changed by anyone:** • See P4.DelayModifier: by P4.DelayModifier.8, the Term multisig holders on the DelayOwnerSafe cannot change the Modules of Delay Modifier, so nobody can.
P4.OwnerlessSafe.8	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.9.
P4.OwnerlessSafe.9	ProposerSafe fallbackHandler is `address(0)`, which cannot be changed by anyone, P4.ProposerSafe.	**Already proven — ProposerSafe fallbackHandler is `address(0)`:** • Same as P1.10. **Already proven — ProposerSafe fallbackHandler cannot be changed by anyone:** • See P4.ProposerSafe: by P4.ProposerSafe.5, the Term multisig holders on the ProposerSafe cannot change the fallbackHandler, so nobody can.
P4.OwnerlessSafe.10	`Modules of ProposerSafe = null`, which cannot be changed by anyone, P4.ProposerSafe.	**Already proven — The Modules of ProposerSafe list is empty:** • Same as P1.12. **Already proven — Modules of ProposerSafe cannot be changed by anyone:** • See P4.ProposerSafe: by P4.ProposerSafe.5, the Term multisig holders on the ProposerSafe cannot change the Modules of ProposerSafe list, so nobody can.
P4.OwnerlessSafe.11	`Owners of ProposerSafe = Term multisig holders on the ProposerSafe`, which can only be changed by Term multisig holders on the ProposerSafe, P4.ProposerSafe.	Same as P4.ProposerSafe.4.

#	Statement	Evidence
P4.OwnerlessSafe.12	The owners for the OwnerlessSafe can be changed by Term multisig holders on the ProposerSafe and Term multisig holders on the OwnerlessSafe; the fallbackHandler, modules, guard, module guard for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe.	**FV — The Term multisig holders on the OwnerlessSafe can change Owners of OwnerlessSafe through `execTransaction`:** • CL130L2-21 / CL141L2-21 / CL150L2-21 `guardedOwnersCanStillChangeOtherSettings` ✅ passes on GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 and SafeL2 v1.5.0, the OwnerlessSafe's possible code (P1.2), for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL130L2-21 / CL141L2-21 / CL150L2-21 stubs the signature check. By P1.2, `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE130L2-5 / SE141L2-5 / SE150L2-5, SE130L2-12 / SE141L2-12 / SE150L2-12), and their approvals pass it (SE130L2-6 / SE141L2-6 / SE150L2-6). **FV — The Term multisig holders on the ProposerSafe can change the owners, fallbackHandler, modules and guard of the OwnerlessSafe, and on SafeL2 v1.5.0 its module guard, through the Delay Modifier:** • PO130L2-5 / PO141L2-5 / PO150L2-5 `proposerApprovalChangesOwnerlessSettings` ✅ passes, for each of `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard` and `setFallbackHandler`, and on SafeL2 v1.5.0 `setModuleGuard`, in one execution: ◦ An owner of the ProposerSafe (a Term multisig holder, P1.11) approves with `approveHash` the hash of the ProposerSafe's call to the Delay Modifier's `execTransactionFromModule(OwnerlessSafe, 0, <settings call>, Call)`, and anyone submits it through `execTransaction` with that approval as the signature. ◦ The ProposerSafe is the Delay Modifier's module (P1.8), so the Delay Modifier queues the proposal. ◦ Once the cooldown has passed and before the proposal expires, anyone's `executeNextTx` hands it to the Delay Modifier's target, the OwnerlessSafe, which has the Delay Modifier as its module (P1.5). ◦ The OwnerlessSafe's `execTransactionFromModule` makes it call its own settings function, and the setting changes. • PO130L2-5 / PO141L2-5 / PO150L2-5 sets the ProposerSafe's threshold to 1. By SE130L2-6 / SE141L2-6 / SE150L2-6 (P1.9), with the real threshold of 5 (P1.11), five owners' approvals take the same path. • PO130L2-5 / PO141L2-5 / PO150L2-5 assumes no guard on either Safe or on the Delay Modifier. The Delay Modifier's guard is the PauseGuard. DP-18 `checkTransactionAcceptsWhileNotPaused` ✅ passes: while not paused, the PauseGuard accepts any transaction. Its `checkAfterExecution` does nothing (`PauseGuard.sol:123`), so with the PauseGuard not paused, PO130L2-5 / PO141L2-5 / PO150L2-5's path is unchanged. • The ProposerSafe's guard is ConfigLockGuard (P4.ProposerSafe.5). The ProposerSafe's call is to the Delay Modifier's `execTransactionFromModule`, which is not one of the guard's locked calls. CL130L2-10 / CL141L2-10 / CL150L2-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so PO130L2-5 / PO141L2-5 / PO150L2-5's ProposerSafe step is unchanged. • The OwnerlessSafe's guard is ConfigLockGuard (below), but it is never consulted on this path: each version consults its guard only in `execTransaction` (GnosisSafeL2 v1.3.0: `GnosisSafe.sol:147–191`; SafeL2 v1.4.1: `Safe.sol:174–218`; SafeL2 v1.5.0: `Safe.sol:149–204`), not in `execTransactionFromModule` (GnosisSafeL2 v1.3.0: `GnosisSafeL2.sol:77–85`, `ModuleManager.sol:61–73`; SafeL2 v1.4.1: `SafeL2.sol:65–73`, `ModuleManager.sol:81–93`; SafeL2 v1.5.0: `ModuleManager.sol:154–163`, which consults only the module guard, `ModuleManager.sol:94–109`). CL130L2-22 / CL141L2-22 / CL150L2-22 `moduleCallBypassesTheGuard` ✅ passes: with ConfigLockGuard installed, an enabled module's `execTransactionFromModule` addressed to the Safe with a `setGuard` call succeeds and moves the guard out of its slot. LG130L2-2 / LG141L2-2 / LG150L2-2 `moduleCanStillChangeEachSetting` ✅ passes: with ConfigLockGuard installed and its `lockedDelay` the Delay Modifier (P4.DelayModifier.8), an `execTransactionFromModule` from the Delay Modifier, as an enabled module, addressed to the Safe with a call to each of `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard` and `setFallbackHandler`, and on SafeL2 v1.5.0 `setModuleGuard`, succeeds and a setting changes. • PO130L2-5 / PO141L2-5 / PO150L2-5 assumes the deployed wiring: the ProposerSafe is a module of the Delay Modifier

#	Statement	Evidence
		(P1.8), and the Delay Modifier is a module of the OwnerlessSafe (P1.5). The Delay Modifier's target is the OwnerlessSafe and its guard is the PauseGuard, read in P1.13. If the OwnerlessSafe runs SafeL2 v1.5.0, its module guard, which would check this path's module transaction, must be `address(0)`, also read in P1.13; PO150L2-5 and LG150L2-2 run with it unset. Each of PO130L2-5 / PO141L2-5 / PO150L2-5 runs both Safes on the same version. • While the PauseGuard is paused, it rejects every `executeNextTx` (`PauseGuard.sol:119`), so the proposal waits until the PauseGuard's admin unpauses it. If `txExpiration` passes first, the proposal expires and has to be proposed again. **FV — The Term multisig holders on the OwnerlessSafe cannot change OwnerlessSafe fallbackHandler, Modules of OwnerlessSafe, OwnerlessSafe guard or, on SafeL2 v1.5.0, OwnerlessSafe module guard through `execTransaction` if OwnerlessSafe guard is ConfigLockGuard, so only the Term multisig holders on the ProposerSafe can change them:** • CL130L2-2 / CL141L2-2 / CL150L2-2 `checkTransactionRejectsSelfSetGuard`, CL130L2-3 / CL141L2-3 / CL150L2-3 `checkTransactionRejectsSelfEnableModule`, CL130L2-4 / CL141L2-4 / CL150L2-4 `checkTransactionRejectsSelfDisableModule`, CL130L2-5 / CL141L2-5 / CL150L2-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL130L2-12 / CL141L2-12 / CL150L2-12 `guardedSafeCannotCallSetGuard`, CL130L2-13 / CL141L2-13 / CL150L2-13 `guardedSafeCannotCallEnableModule`, CL130L2-14 / CL141L2-14 / CL150L2-14 `guardedSafeCannotCallDisableModule`, CL130L2-15 / CL141L2-15 / CL150L2-15 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL130L2-17 / CL141L2-17 / CL150L2-17 `guardedSelfCallNeverChangesGuard`, CL130L2-18 / CL141L2-18 / CL150L2-18 `guardedSelfCallNeverChangesModules`, CL130L2-19 / CL141L2-19 / CL150L2-19 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of OwnerlessSafe list and the fallbackHandler unchanged. • CL150L2-25 `checkTransactionRejectsSelfSetModuleGuard`, CL150L2-26 `guardedSafeCannotCallSetModuleGuard`, CL150L2-27 `guardedSelfCallNeverChangesModuleGuard` ✅ pass (SafeL2 v1.5.0 only): called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setModuleGuard`; with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with that call reverts; and for any calldata and operation, an `execTransaction` addressed to the Safe leaves the module guard unchanged. • CL130L2-1 / CL141L2-1 / CL150L2-1 `checkTransactionRejectsDelegateCall`, CL130L2-16 / CL141L2-16 / CL150L2-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of OwnerlessSafe list, the fallbackHandler or, on SafeL2 v1.5.0, the module guard directly. • By P1.2, these settings only change when the caller is the Safe itself (SE130L2-7 / SE141L2-7 / SE150L2-7), so an `execTransaction` to any other address cannot change them either. • These rules run on GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 and SafeL2 v1.5.0, the OwnerlessSafe's possible code (P1.2), and leave `lockedDelay` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P4.OwnerlessSafe.2 and P4.OwnerlessSafe.4, only Owners of OwnerlessSafe or Modules of OwnerlessSafe can change the fallbackHandler and the Modules of OwnerlessSafe list, and SC130L2-4 / SC141L2-4 / SC150L2-4 (P4.OwnerlessSafe.2) covers the guard, and on SafeL2 v1.5.0 the module guard, the same way. The only Module of OwnerlessSafe is the Delay Modifier (P4.OwnerlessSafe.4), which only runs transactions queued by Modules of Delay Modifier (P4.OwnerlessSafe.5, P4.OwnerlessSafe.6). The only Module of Delay Modifier is the ProposerSafe, which cannot be changed by anyone (P4.OwnerlessSafe.7), and only the Term multisig holders on the ProposerSafe can make the ProposerSafe act (P4.OwnerlessSafe.8 to P4.OwnerlessSafe.11). So only the Term multisig holders on the ProposerSafe can change them.

#	Statement	Evidence
		On-chain — OwnerlessSafe guard is ConfigLockGuard: • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:30` / `:42` / `:66` in GnosisSafeL2 v1.3.0 / SafeL2 v1.4.1 / SafeL2 v1.5.0) on the OwnerlessSafe must hold ConfigLockGuard. • Assumes the OwnerlessSafe runs GnosisSafeL2 v1.3.0, SafeL2 v1.4.1 or SafeL2 v1.5.0 (P1.2), the versions the CL, LG and PO rules above check.

Proposition 5 — The owners, fallbackHandler, modules, guard for the PauseSafe can be changed only by the Term multisig holders on the PauseSafe. The pauser for the PauseGuard can be changed only by the Term multisig holders on the ProposerSafe. The admin for the PauseGuard cannot be changed by anyone.

PauseSafe

P5.PauseSafe: The owners, fallbackHandler, modules, guard for the PauseSafe can only be changed by Term multisig holders on the PauseSafe.

#	Statement	Evidence
P5.PauseSafe.1	Only Modules of PauseSafe and Owners of PauseSafe can interact with the PauseSafe, except `execTransaction`, which anyone can call only if Owners of PauseSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P3.Pause.3.
P5.PauseSafe.2	PauseSafe fallbackHandler is `address(0)`, which can only be changed by Owners of PauseSafe and Modules of PauseSafe.	Same as P3.Pause.4. **FV — PauseSafe fallbackHandler can only be changed by Owners of PauseSafe or Modules of PauseSafe:** • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set, the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P5.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150L2-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE150L2-5, SE150L2-12). So only Modules of PauseSafe or Owners of PauseSafe can change the fallbackHandler.
P5.PauseSafe.3	`Modules of PauseSafe = null`, which can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`.	Same as P3.Pause.6. **FV — Modules of PauseSafe can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`:** • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set, the module list only changes through an enabled module's `execTransactionFromModule`

#	Statement	Evidence
		or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P5.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150L2-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE150L2-5, SE150L2-12). • The PauseSafe has no module (P3.Pause.6), so only the Owners of PauseSafe can change the module list.
P5.PauseSafe.4	`Owners of PauseSafe = Term multisig holders on the PauseSafe`, which can only be changed by Term multisig holders on the PauseSafe because `Modules of PauseSafe = null`.	Same as P3.Pause.5. **FV — Owners of PauseSafe can only be changed by Term multisig holders on the PauseSafe because `Modules of PauseSafe = null`:** • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set, the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P5.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150L2-3), and `execTransaction` only acts once two Owners of PauseSafe, the Term multisig holders (P3.Pause.5), have approved (SE150L2-5, SE150L2-12). • The PauseSafe has no module (P3.Pause.6), so only the Term multisig holders on the PauseSafe can change the owners and threshold.
P5.PauseSafe.5	PauseSafe guard is `address(0)`, which can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`.	**On-chain — PauseSafe guard is `address(0)`:** • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in SafeL2 v1.5.0) on the PauseSafe must hold `address(0)`. • Assumes the PauseSafe runs SafeL2 v1.5.0, the version SE150L2 and SC150L2 check (`GuardManager.sol:66`). **FV — PauseSafe guard can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`:** • SC150L2-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set, the guard only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P5.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150L2-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE150L2-5, SE150L2-12). • The PauseSafe has no module (P3.Pause.6), so only the Owners of PauseSafe can change the guard.
P5.PauseSafe.6	The owners, fallbackHandler, modules, guard for the PauseSafe can be changed by Term multisig holders on the PauseSafe.	**FV — The Term multisig holders on the PauseSafe can change the owners, fallbackHandler, modules and guard for the PauseSafe through `execTransaction`:** • SC150L2-3 `ownersCanChangeSettings` ✅ passes: for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `enableModule`, `disableModule`, `setGuard` and `setFallbackHandler`, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. • SE150L2-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. The PauseSafe runs SafeL2 v1.5.0 (P3.Pause.5), and its owners are the Term multisig holders on the PauseSafe (P3.Pause.5). • The PauseSafe has no guard (P5.PauseSafe.5), so nothing rejects the owners' `execTransaction`.

PauseGuard

P5.PauseGuard: The pauser for the PauseGuard can only be changed by Term multisig holders on the ProposerSafe; the admin for the PauseGuard cannot be changed by anyone.

#	Statement	Evidence
P5.PauseGuard.1	Only the PauseGuard pauser and the PauseGuard admin can interact with the PauseGuard, except `checkTransaction` and `checkAfterExecution`, which anyone can call but which do not change state.	Same as P3.1.
P5.PauseGuard.2	`PauseGuard admin = ProposerSafe`, which cannot be changed by anyone.	Same as P3.Admin.2. **FV — Nobody can change the PauseGuard admin:** • DP-14 `adminNeverChanges` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, the admin never changes. • `admin` is `immutable` (`PauseGuard.sol:17`), set once in the constructor, so no function can write it.
P5.PauseGuard.3	`PauseGuard pauser = PauseSafe`, which can only be changed by Term multisig holders on the ProposerSafe.	Same as P3.Pause.2. **FV — The PauseGuard pauser can only be changed by Term multisig holders on the ProposerSafe:** • DP-13 `pauserOnlyChangesThroughSetPauser` ✅ passes: over every write function of both contracts, the pauser only changes through `setPauser`. • DP-10 `setPauserRevertsForAnyoneButTheAdmin` ✅ passes: anyone who is not the admin is rejected. • The PauseGuard admin is the ProposerSafe (P5.PauseGuard.2), and only the Term multisig holders on the ProposerSafe can make it act (P4.ProposerSafe.1, P4.ProposerSafe.3, P4.ProposerSafe.4). So only they can change the pauser.
P5.PauseGuard.4	The pauser for the PauseGuard can be changed by Term multisig holders on the ProposerSafe; the admin for the PauseGuard cannot be changed by Term multisig holders on the ProposerSafe, so it cannot be changed by anyone.	**FV — The Term multisig holders on the ProposerSafe can change the PauseGuard pauser:** • DP-11 `setPauserSetsThePauser` ✅ passes: for any nonzero account, the admin's `setPauser(account)` succeeds, `pauser()` then returns that account, and the previous pauser no longer holds the slot. • SE130L2-6 / SE141L2-6 / SE150L2-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the ProposerSafe's owners (P3.Admin.5) can have it call `setPauser`. • The ProposerSafe's guard is ConfigLockGuard (P4.ProposerSafe.5). Its call to `setPauser` on the PauseGuard is a plain call to neither the ProposerSafe nor its locked Delay Modifier, so it is not a locked call. CL130L2-10 / CL141L2-10 / CL150L2-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so the call is unchanged. **FV — The Term multisig holders on the ProposerSafe cannot change the PauseGuard admin, so nobody can:** • Same as P5.PauseGuard.2: the admin never changes, whoever the caller is, the ProposerSafe included.

4. Appendix

4.1 Formal Verification

All Certora specs and confs for this document: `packages/evm/certora/2.1.0`.

Each rule cited in Section 3 is named by a prefix and a number, such as SE141L2-5 or DM-13. The table below lists each prefix, what its rules cover, which of its rules Section 3 uses, the spec that defines them and the conf that runs them. A prefix whose rules are spread over more than one spec has one row per spec. A rule that passes on every Safe version a statement covers is cited once per version, joined by slashes, as in SE130L2-6 / SE141L2-6 / SE150L2-6. A rule cited alone, such as SE130L2-10 or CL150L2-25, is checked on that version only.

Prefix	What it covers	Rules used in Section 3	Spec	Conf
SE141L2	SafeL2 v1.4.1 entry points and who can make the Safe act	SE141L2-1, SE141L2-3 to SE141L2-9, SE141L2-11, SE141L2-12	`SafeL2V141/executionPaths.spec`	`Safe-L2-v141-executionPaths.conf`
	SafeL2 v1.4.1 `fallback` and `receive` change no state	SE141L2-21 to SE141L2-26	`SafeL2V141/noStateChange.spec`	`Safe-L2-v141-noStateChange.conf`
SC141L2	SafeL2 v1.4.1 settings change only through a module or the owners	SC141L2-1 to SC141L2-4	`SafeL2V141/selfCalls.spec`	`Safe-L2-v141-selfCalls.conf`
SE130L2	GnosisSafeL2 v1.3.0 entry points and who can make the Safe act	SE130L2-1, SE130L2-3 to SE130L2-12	`SafeL2V130/executionPaths.spec`	`Safe-L2-v130-executionPaths.conf`
	GnosisSafeL2 v1.3.0 `fallback` and `receive` change no state	SE130L2-21 to SE130L2-26	`SafeL2V130/noStateChange.spec`	`Safe-L2-v130-noStateChange.conf`
SC130L2	GnosisSafeL2 v1.3.0 settings change only through a module or the owners	SC130L2-1 to SC130L2-4	`SafeL2V130/selfCalls.spec`	`Safe-L2-v130-selfCalls.conf`
SE150L2	SafeL2 v1.5.0 entry points and who can make the Safe act	SE150L2-1, SE150L2-3 to SE150L2-9, SE150L2-11, SE150L2-12	`SafeL2V150/executionPaths.spec`	`Safe-L2-v150-executionPaths.conf`
	SafeL2 v1.5.0 `fallback` and `receive` change no state	SE150L2-21 to SE150L2-26	`SafeL2V150/noStateChange.spec`	`Safe-L2-v150-noStateChange.conf`
SC150L2	SafeL2 v1.5.0 settings change only through a module or the owners	SC150L2-1 to SC150L2-4	`SafeL2V150/selfCalls.spec`	`Safe-L2-v150-selfCalls.conf`
LG141L2	ConfigLockGuard on the OwnerlessSafe, run on SafeL2 v1.4.1: the module path	LG141L2-2, LG141L2-3	`SafeL2V141/ownerlessSafeGuard.spec`	`Safe-L2-v141-ownerlessSafeGuard.conf`
LG130L2	ConfigLockGuard on the OwnerlessSafe, run on GnosisSafeL2 v1.3.0: the module path	LG130L2-2, LG130L2-3	`SafeL2V130/ownerlessSafeGuard.spec`	`Safe-L2-v130-ownerlessSafeGuard.conf`
LG150L2	ConfigLockGuard on the OwnerlessSafe, run on SafeL2 v1.5.0: the module path	LG150L2-2, LG150L2-3	`SafeL2V150/ownerlessSafeGuard.spec`	`Safe-L2-v150-ownerlessSafeGuard.conf`

Prefix	What it covers	Rules used in Section 3	Spec	Conf
CL141L2	ConfigLockGuard on a SafeL2 v1.4.1 Safe: what owners cannot change on the Safe and its locked modifier, what owners can still do, and the module path it does not check	CL141L2-1 to CL141L2-10, CL141L2-12 to CL141L2-19, CL141L2-21, CL141L2-22, CL141L2-24	`SafeL2V141/configLockGuard.spec`	`Safe-L2-v141-configLockGuard.conf`
CL130L2	ConfigLockGuard on a GnosisSafeL2 v1.3.0 Safe: what owners cannot change on the Safe and its locked modifier, what owners can still do, and the module path it does not check	CL130L2-1 to CL130L2-10, CL130L2-12 to CL130L2-19, CL130L2-21, CL130L2-22, CL130L2-24	`SafeL2V130/configLockGuard.spec`	`Safe-L2-v130-configLockGuard.conf`
CL150L2	ConfigLockGuard on a SafeL2 v1.5.0 Safe: what owners cannot change on the Safe, its module guard included, and its locked modifier, what owners can still do, and the module path it does not check	CL150L2-1 to CL150L2-10, CL150L2-12 to CL150L2-19, CL150L2-21 to CL150L2-27	`SafeL2V150/configLockGuard.spec`	`Safe-L2-v150-configLockGuard.conf`
OD150L2	ConfigLockGuard locking the Delay Modifier: no unlocked call changes its owner, target, modules, guard, txCooldown or txExpiration, and the DelayOwnerSafe, on SafeL2 v1.5.0, can still veto	OD150L2-1, OD150L2-2, OD150L2-4, OD150L2-5	`Delay/SafeL2V150/delayOwnerGuard.spec`	`Delay-SafeL2-v150-delayOwnerGuard.conf`
DM	Delay Modifier access control and module list	DM-1, DM-2, DM-4 to DM-8, DM-11 to DM-13, DM-15 to DM-20	`Delay/moduleIntegrity.spec`	`Delay-moduleIntegrity.conf`
DQ	Delay Modifier queue provenance	DQ-2 to DQ-6	`Delay/queueProvenance.spec`	`Delay-queueProvenance.conf`
DE141L2	DEVOPS_ROLE methods end to end, with the ProposerSafe and OwnerlessSafe on SafeL2 v1.4.1	DE141L2-1, DE141L2-2	`Delay/SafeL2V141/devopsEndToEnd.spec`	`Delay-SafeL2-v141-devopsEndToEnd.conf`
DE130L2	DEVOPS_ROLE methods end to end, with the ProposerSafe and OwnerlessSafe on GnosisSafeL2 v1.3.0	DE130L2-1, DE130L2-2	`Delay/SafeL2V130/devopsEndToEnd.spec`	`Delay-SafeL2-v130-devopsEndToEnd.conf`
DE150L2	DEVOPS_ROLE methods end to end, with the ProposerSafe and OwnerlessSafe on SafeL2 v1.5.0	DE150L2-1, DE150L2-2	`Delay/SafeL2V150/devopsEndToEnd.spec`	`Delay-SafeL2-v150-devopsEndToEnd.conf`
DV150L2	DelayOwnerSafe owners veto directly, with the DelayOwnerSafe on SafeL2 v1.5.0	DV150L2-1	`Delay/SafeL2V150/vetoSignedPath.spec`	`Delay-SafeL2-v150-vetoSignedPath.conf`
DP	PauseGuard, and the Delay Modifier's veto window and vetoes	DP-3 to DP-18, DP-24 to DP-33	`Delay/pauseGuardSufficient.spec`	`Delay-pauseGuardSufficient.conf`
PO141L2	ProposerSafe → Delay Modifier → OwnerlessSafe settings, with both Safes on SafeL2 v1.4.1	PO141L2-5	`Delay/SafeL2V141/proposerToOwnerlessSettings.spec`	`Delay-SafeL2-v141-proposerToOwnerlessSettings.conf`
PO130L2	ProposerSafe → Delay Modifier → OwnerlessSafe settings, with both Safes on GnosisSafeL2 v1.3.0	PO130L2-5	`Delay/SafeL2V130/proposerToOwnerlessSettings.spec`	`Delay-SafeL2-v130-proposerToOwnerlessSettings.conf`

Prefix	What it covers	Rules used in Section 3	Spec	Conf
PO150L2	ProposerSafe → Delay Modifier → OwnerlessSafe settings, with both Safes on SafeL2 v1.5.0	PO150L2-5	`Delay/SafeL2V150/proposerToOwnerlessSettings.spec`	`Delay-SafeL2-v150-proposerToOwnerlessSettings.conf`

4.2 On-chain

All contracts are on Ethereum mainnet. Each address links to its Etherscan page. Contracts marked to be deployed are part of the proposed 2.1.0 deployment and have no address yet. Contracts marked TBD have no address decided yet.

Safes

Contract	Address
OwnerlessSafe	to be deployed
ProposerSafe	to be deployed
DelayOwnerSafe	to be deployed
PauseSafe	to be deployed
GnosisSafeL2 v1.3.0 mastercopy	`0x3E5c63644E683549055b9Be8653de26E0B4CD36E`
SafeL2 v1.4.1 mastercopy	`0x29fcB43b46531BcA003ddC8FCB67FFE91900C762`
SafeL2 v1.5.0 mastercopy	`0xEdd160fEBBD92E350D4D398fb636302fccd67C7e`

Modifiers

Contract	Address
Delay Modifier	to be deployed

Guards

Contract	Address
PauseGuard	to be deployed
ConfigLockGuard	to be deployed

Term Protocol contracts

Contract	Address
TermController	TBD
TermEventEmitter	TBD
TermPriceConsumerV3	TBD
TermRepoDeployerFactory	TBD
TermInitializer	TBD
TermDiamond	TBD